PRODUCT • FUNCTIONAL • TECHNICAL

Audiobook Production Platform Revamp Blueprint

Requirements, Functional Specification, Technical Architecture, Delivery Roadmap, and Prosperity Operating Model

FROM FUNCTIONAL PROTOTYPE TO PUBLISHING-GRADE PRODUCTION STUDIO

Document	Value
Prepared from	fullstack_project source snapshot
Audit date	July 23, 2026
Document status	Implementation-ready working specification
Recommended approach	Evolutionary rebuild; preserve domain strengths, replace unsafe runtime boundaries

This document intentionally goes beyond repository cleanup. It defines the product that can earn trust, retain users, and scale.

Document Control and Decision Summary

This blueprint is the controlling product and engineering specification for revamping the current audiobook tool. It combines a current-state audit, a product requirements document, a functional specification, a technical design, a delivery plan, and a commercial operating model. Where implementation choices remain open, this document states a recommended default and the evidence required to choose differently.

> **Executive decision:** Do not discard the prototype, and do not scale it as-is. Preserve its provider contract, chapter-aware processing, content-addressed reuse idea, MP3/M4B assembly knowledge, and progress-oriented UI. Replace its in-memory jobs, local-only storage, anonymous access, permissive upload boundary, hardcoded deployment assumptions, and warning-only audio checks.

Decision area	Recommended decision	Disposition
Business posture	Narrow initial wedge, professional workflow, measurable output quality	Approve
Architecture	Modular monolith plus durable background workers; no premature microservices	Approve
Data	PostgreSQL system of record; object storage for manuscripts and audio; Redis/SQS coordination	Approve
Security	Authenticated multi-tenant workspace, least privilege, signed assets, retention and audit controls	Approve
Audio	Versioned synthesis, mastering, and export profiles with objective QC and human approval	Approve
Provider strategy	Commercially authorized providers and BYOK; Edge read-aloud only as a local/development adapter	Approve
Delivery	Repository rescue first; then production foundation; then studio workflow; then growth features	Approve

Revision rule: changes to requirements, state models, data ownership, provider policy, security posture, or export profiles should be recorded as explicit Architecture Decision Records (ADRs) and reflected in acceptance tests. Small UI refinements do not require revision of this governing document.

Contents

Executive Blueprint

The honest verdict

The current code is a credible proof that the core pipeline can work: manuscript ingestion, chapter detection, provider abstraction, chunk synthesis, reuse of unchanged work, chapter assembly, basic QC, progress reporting, and MP3/M4B output are all represented in executable code. That is valuable. It means the team is not starting from a slide deck.

But the application is not yet a product customers can safely entrust with manuscripts, budgets, deadlines, or deliverables. The gap is not one missing feature. The runtime has no durable job record, no user or organization model, no ownership checks, no restart recovery, no concurrency governance, no cost ledger, no production storage boundary, no approval workflow, and no platform-grade mastering. The frontend is a single-session converter. The repository itself is not cleanly reproducible.

> **Prosperity thesis:** The winning product is not 'text in, MP3 out.' That feature is easily copied. The defensible product is a revision-safe production system that makes long-form audio predictable: structured manuscripts, explainable voice decisions, controlled spend, resumable work, targeted regeneration, objective QC, human approvals, and retailer-ready evidence.

Current-to-target maturity

Dimension	Today	Evidence	Target
Product concept	8/10	Real workflow and useful domain choices	Turn the converter into a persistent studio
Prototype functionality	7/10	Core happy path is implemented	Add editing, previews, recovery, approvals, exports
Repository hygiene	4/10	Source mixed with generated artifacts; CI misplaced	Clean, reproducible, governed monorepo
Deployment readiness	3/10	Development servers and local paths dominate	Immutable images, health, migrations, storage
Security and tenancy	1/10	Anonymous global process with task-ID access	Organizations, RBAC, isolation, audit, retention
Operational resilience	2/10	Daemon threads and in-memory state	Durable queue, retries, cancellation, reconciliation
Audio compliance	3/10	Basic warnings exist	Mastering profiles, measurable gates, per-target packages
Commercial readiness	2/10	No plans, quotas, ledger, or policy controls	Usage governance, margin visibility, packaging

What must happen in order

1. Make the repository reproducible: correct the Vite entry point and CI location, remove generated junk, add ignore rules and lockfiles, and prove clean builds.
2. Protect the product boundary: authenticate users, isolate organizations, validate uploads, move secrets out of files, and stop serving assets by possession of a task identifier.
3. Make work durable: persist projects and jobs, use a real queue, write artifacts to object storage, support retry/cancel/resume, and reconcile interrupted work.
4. Create the studio workflow: project dashboard, manuscript versions, chapter editor, sample lab, pronunciation dictionary, cost estimate, per-chapter generation, and selective regeneration.
5. Create trustworthy output: versioned mastering profiles, measurable audio gates, review checkpoints, cover/metadata handling, delivery manifests, and platform-specific packages.
6. Install the business controls: quotas, usage ledger, plan entitlements, billing hooks, retention, provider policy, unit economics, activation metrics, and support tooling.
7. Scale only after evidence: measure conversion, completion, cost, QC pass rate, rework, retention, support burden, and gross margin before splitting into microservices or adding speculative AI features.

North-star outcomes

Outcome	Initial target	Why it matters
Time to first playable sample	≤ 10 minutes after a valid upload for a normal chapter	Activation
Project recovery	100% of accepted jobs survive API/worker restart without silent loss	Trust
Selective regeneration	Editing one segment regenerates only invalidated downstream artifacts	Cost/time
Estimate accuracy	Final provider charge within ±10% of approved estimate absent user changes	Cost control
QC transparency	Every deliverable has machine-readable measurements and human approval state	Quality
Supportability	Every failed job has a user-safe message, traceable error code, and retry path	Operations
Tenant isolation	Zero cross-organization asset, cache, task, or metadata access in automated tests	Security
Release confidence	Critical path covered by unit, integration, contract, audio-fixture, and end-to-end tests	Velocity
Commercial signal	≥ 40% of activated projects reach an approved sample; ≥ 25% reach an export milestone	Product-market learning

Current-State Audit

Audit basis

The audit reviewed the Drive-hosted fullstack_project snapshot, including the root configuration, Flask API, file and text processing services, provider adapters, audio utilities, tests, React/TypeScript frontend, Dockerfiles, Compose file, and the stored GitHub Actions definition. Conclusions below distinguish verified code behavior from recommended future behavior.

Primary source: fullstack_project README

Strengths to preserve

- Provider abstraction: SpeechProvider and ProviderRegistry keep vendor SDK details out of the main pipeline.
- Structured DOCX ingestion: Heading 1/2 paragraphs are promoted to explicit chapter markers rather than relying only on regex guesses.
- Content-addressed reuse: synthesis chunks are hashed by provider, voice, engine, and text, and writes are atomic.
- Chapter-aware execution: the pipeline produces chapter files, tracks chapter progress, and assembles both a merged MP3 and chapterized M4B.
- Useful visibility: the UI exposes current chapter, cached versus newly synthesized chunks, chapter duration, and warning summaries.
- Optional cloud provider: Polly is inactive when credentials are missing rather than preventing local startup.
- Reasonable dependency pinning on the backend and strict TypeScript configuration on the frontend.
- Tests cover health, providers, chapter markers and heuristics, chunk limits, paragraph preservation, cache round trips, and empty synthesis input.

Verified blocker and risk ledger

ID	Severity	Area	Verified condition	Required correction
AUD-001	Critical	Build	Vite index.html is under frontend/public; Vite expects the application entry at frontend/index.html.	Move/create the root entry and make a clean frontend build a release gate.
AUD-002	Critical	CI	github-actions.yml is stored under ci-cd; GitHub only runs workflows under .github/workflows.	Move to .github/workflows/ci.yml and add required status checks.
AUD-003	High	Hygiene	.DS_Store, __pycache__, .pytest_cache, cache, outputs, and uploads are present in the source snapshot.	Remove generated artifacts and enforce .gitignore/.dockerignore.

ID	Severity	Area	Verified condition	Required correction
AUD-004	High	Frontend	API_BASE_URL is hardcoded to http://localhost:5000/api.	Use VITE_API_BASE_URL or same-origin routing with environment validation.
AUD-005	Critical	Jobs	active_tasks is an in-memory dictionary and jobs run in daemon threads.	Persist jobs; enqueue durable work; recover after restart.
AUD-006	Critical	Tenancy	There is no authentication, organization model, ownership check, or role model.	Require authenticated organization context on every protected resource.
AUD-007	Critical	Authorization	Task IDs function as bearer capabilities for status and downloads.	Authorize by project/org; issue short-lived signed asset links.
AUD-008	High	Capacity	Every synthesis request can create an unbounded daemon thread.	Queue jobs, cap concurrency by provider/plan/org, and apply backpressure.
AUD-009	High	Recovery	No cancel, pause, retry, resume, heartbeat, lease, or orphan-job reconciliation exists.	Implement explicit job/attempt state machines and scheduled reconciliation.
AUD-010	High	Cost	Paid synthesis can start without a cost estimate, budget approval, quota, or ledger.	Preflight billable characters; enforce budget and record actual usage.
AUD-011	Critical	Upload	The backend does not allowlist extensions or verify MIME/content signatures.	Use MIME sniffing, extension/content agreement, malware scan, and parser sandbox.
AUD-012	High	Upload	A 100 MB request limit exists, but parsing is synchronous and no decompression or parser resource budget exists.	Use presigned uploads and asynchronous bounded ingestion with time/memory caps.
AUD-013	High	Privacy	The upload endpoint returns the complete extracted manuscript to the browser.	Return structured summaries and paged text; minimize exposure and logging.
AUD-014	High	Storage	Uploads, cache, and outputs use local relative directories; Compose persists only uploads.	Use object storage, lifecycle rules, and durable artifact metadata.
AUD-015	High	Cache	Cache keys omit provider/model version, synthesis parameters, normalization version, SSML, and output profile.	Hash the canonical segment plus the complete versioned synthesis configuration.
AUD-016	High	Privacy	The cache is global rather than tenant/project scoped.	Default to organization-scoped reuse; make cross-tenant dedupe a separate policy decision.
AUD-017	High	Reliability	Provider docstring says the pipeline handles retries, but synthesis calls have no retry/backoff or error classification.	Use retryable/nonretryable errors, exponential backoff with jitter, and attempt caps.
AUD-018	High	Provider	Microsoft Edge read-aloud is presented as the free default for personal use.	Treat it as local/dev unless commercial terms are explicitly approved; use licensed APIs in hosted plans.
AUD-019	Medium	Provider	Voice availability is checked largely by import/credential presence, not a bounded live health probe.	Track provider health, latency, quota, model support, and degradation state.
AUD-020	High	Text	Unknown uploads fall through to plaintext decoding, including binary content.	Reject unsupported formats before parsing.
AUD-021	Medium	Text	DOCX Title style becomes a chapter, potentially turning the book title into content.	Distinguish document title, front matter, parts, chapters, and sections.
AUD-022	Medium	Text	Heuristics can mistake standalone numbers, Roman numerals, or separators for chapters.	Show confidence and require user confirmation/editing before production.
AUD-023	Medium	Text	Pathological long sentences are split by character count, potentially cutting words and pronunciations.	Use token/word-aware segmentation with provider-specific SSML limits.

ID	Severity	Area	Verified condition	Required correction
AUD-024	High	Editing	The system has no persisted manuscript version, chapter editor, diff, or selective invalidation graph.	Make source and derived artifacts versioned and immutable.
AUD-025	High	Audio	merge_audio_files decodes and re-encodes MP3s at 128 kbps.	Use loss-minimizing intermediates and explicit mastering/export profiles.
AUD-026	Critical	Audio QC	QC flags only extreme quiet, near-zero peaks, long silence, and sub-second audio.	Measure target-specific RMS/LUFS, true/sample peak, noise, head/tail, format, channels, sample rate, bitrate, duration, and consistency.
AUD-027	High	Audio	normalize_audio exists but is not integrated; no deterministic mastering chain exists.	Implement versioned mastering with measured before/after results.
AUD-028	Medium	Packaging	M4B includes title/author/chapter markers but no cover art, narrator, publisher, ISBN, language, year, or manifest.	Create validated metadata and cover pipelines with export manifests.
AUD-029	Medium	Packaging	The default output is a single merged MP3 plus M4B; retailer chapter-file packages are absent.	Support target profiles and one-file-per-section delivery.
AUD-030	High	Observability	Failures use print statements; there are no structured logs, traces, metrics, alerts, or correlation IDs.	Instrument API, queue, provider calls, FFmpeg, cost, and QC stages.
AUD-031	High	Deployment	The Flask development server and Vite preview server are used as container commands.	Use production WSGI/ASGI and static hosting/CDN or reverse proxy.
AUD-032	High	Containers	Containers run as root and have no health checks, init handling, read-only root filesystem, or resource limits.	Harden images, drop privileges, and define liveness/readiness/resource policy.
AUD-033	Medium	Dependencies	Frontend dependencies use ranges and no lockfile appears in the snapshot.	Commit one lockfile and use deterministic installs.
AUD-034	High	Testing	No frontend unit/component tests, integration environments, browser E2E, provider contract tests, or audio fixture tests exist.	Create a risk-based test pyramid and golden audio fixtures.
AUD-035	High	Quality	No linting, formatting, static type checks for Python, security scanning, SBOM, secret scan, or image scan exists.	Make all quality/security gates visible in CI.
AUD-036	Medium	API	Input enums and arrays are not schema-validated; unsupported formats/engines can travel deep into jobs.	Use typed request schemas, OpenAPI, and stable error codes.
AUD-037	Medium	API	Polling every two seconds is the only status transport and failures are merely logged in the browser.	Use resilient polling initially, then SSE/WebSocket; surface disconnect/retry states.
AUD-038	Medium	UX	The app has one page, one transient task, browser alerts, and no project history.	Build persistent project navigation, nonblocking notifications, drafts, and resumable sessions.
AUD-039	Medium	Accessibility	Interactive cards and custom controls lack demonstrated keyboard/focus/screen-reader behavior.	Target WCAG 2.2 AA and test keyboard, focus, labels, contrast, and live regions.
AUD-040	High	Governance	No rights declaration, voice consent record, synthetic narration disclosure, or target-channel policy exists.	Capture provenance and prevent incompatible distribution packages.

What not to do

- Do not call repository cleanup a production launch. It fixes reproducibility, not customer safety.
- Do not rewrite everything into microservices. The team needs strong boundaries and durable jobs, not distributed-system overhead.
- Do not make a free, unofficial, or consumer-facing speech route the commercial foundation without explicit licensing review.
- Do not add multiple providers before cost, retry, provenance, and contract tests exist; every adapter multiplies operational states.
- Do not market warning-only QC as retailer compliance. A pass/fail claim must be tied to a versioned target profile and measured file.
- Do not allow 'AI magic' to bypass user confirmation of chapters, pronunciations, rights, or spend.

Product Vision and Prosperity Strategy

Recommended positioning

> **Positioning:** A production operating system for turning long-form manuscripts into controlled, reviewable, distribution-ready audio - with cost certainty, revision safety, and objective quality evidence.

The initial market should not be every listener or every publisher. The sharpest wedge is independent authors, small presses, and internal publishing teams with completed text, limited audio operations, repeated backlist opportunities, and a need to produce or evaluate digital narration efficiently. These users feel the cost of rework immediately and value a documented workflow.

Primary personas and jobs to be done

Persona	Core job	Current pain	Product response
Independent rights holder	Produce a credible audiobook without becoming an audio engineer	Budget surprise; rejected files; confusing voices; expensive reruns	Guided project, estimate, sample approval, export profile
Small-press producer	Run multiple titles with consistent standards and clear accountability	Scattered files; inconsistent metadata; no approvals; no portfolio view	Organizations, templates, roles, queues, reusable lexicons, reporting
Editor / proof-listener	Find and correct audible problems without regenerating the whole book	No text-audio linkage; comments lost; full-book reruns	Segment playback, anchored issues, selective regeneration, approvals
Audio operations lead	Keep jobs reliable, compliant, and within provider limits	Opaque failures; throttling; no cost ledger; no retry control	Provider health, attempt logs, budgets, deterministic profiles
Administrator / finance	Control access, retention, vendor spend, and plan entitlements	Anonymous usage; no ownership; no invoice evidence	RBAC, audit, quotas, usage ledger, retention policy

Product principles

1. Project before job. A user owns a persistent production project; generation jobs are traceable activities within it.
2. Preview before spend. No full-book paid run should begin without a playable sample, estimate, and explicit approval.
3. Immutable inputs, versioned derivatives. Every output points to the exact manuscript, chapter text, provider/model, voice, pronunciation set, mastering profile, and software version.
4. Regenerate the smallest safe unit. Edits invalidate only affected segments and downstream assemblies.

5. Quality is evidence. Measurements, warnings, approvals, and waivers travel with the deliverable.
6. Providers are replaceable; projects are durable. A vendor outage or model retirement must not erase editorial work.
7. Safe defaults with visible escape hatches. Advanced SSML and mastering are available without overwhelming first-time users.
8. Commercial policy is code. Entitlements, budgets, voice rights, synthetic narration disclosure, and distribution compatibility are enforced, not buried in a help page.
9. Human approval remains authoritative. Automation proposes structure and flags defects; people approve chapters, voice direction, and release.

Product scope

Scope band	Included capability
MVP must	Projects, organizations, upload/versioning, chapter confirmation, voice preview, pronunciation entries, estimate, durable generation, selective retry, objective QC, review, MP3/M4B/chapter exports, usage ledger
MVP should	Comments, approval roles, templates, webhooks/email, cover/metadata validation, portfolio dashboard, BYOK credentials
Later	Multi-cast dialogue, automated speaker attribution, localization, distribution APIs, advanced mastering marketplace, narrator collaboration, enterprise SSO/SCIM
Not now	Consumer listening app, ebook storefront, generalized DAW, open voice-cloning marketplace, microservice fleet, guaranteed retailer acceptance

Business model hypotheses

Pricing must be validated with customers and provider economics; the figures below are hypotheses, not promises. The model should separate platform value from pass-through synthesis so gross margin remains visible.

Offer	Price hypothesis	Value boundary	Best fit
Creator	$29-$59/month + usage	1 seat, limited active projects, standard profiles, organization-scoped cache	Indie authors testing repeat use
Studio	$149-$349/month + usage	5-15 seats, approvals, templates, higher concurrency, portfolio reporting, webhooks	Small presses and production teams
Enterprise	Annual contract	SSO/SCIM, contractual retention, dedicated limits, audit exports, support SLA, private networking options	Publishers and institutions

Offer	Price hypothesis	Value boundary	Best fit
Service add-ons	Per title or per hour	Human proof-listening, mastering review, metadata cleanup, migration	Revenue plus learning during early market

> **Margin rule:** Record provider cost, storage, compute, egress, support touches, and refunds per project. Never hide usage cost inside a flat plan until the distribution of book length and regeneration behavior is understood.

The moat to build

- A version graph connecting manuscript text, pronunciation decisions, audio segments, QC findings, approvals, and final packages.
- Provider-neutral production recipes that can move a project without losing editorial direction.
- A growing library of target export profiles, deterministic checks, and explainable remediation.
- Team workflow: anchored review, selective regeneration, role approvals, and repeatable templates.
- Operational data about what causes cost, delay, and rejection - converted into better estimates and safer defaults.
- Trust: private manuscripts, explicit retention, clear provenance, and evidence that the right voice and rights were used.

External market and platform realities

Platform policy is not uniform. Spotify for Authors states that it accepts digital-voice audiobooks for Spotify-only distribution, while other channels may impose different narration or rights rules. The product must therefore generate target-specific eligibility guidance and disclosures rather than presenting one universal 'publish everywhere' status.

Surface	Reality	Required product behavior
Spotify for Authors	Direct self-publishing and digital-voice acceptance for Spotify-only distribution	Capture synthetic narration disclosure; separate wide-distribution eligibility
ACX/Audible	Submission and narration requirements are channel-specific and may change	Treat profile as versioned; verify live policy before export
Apple Books	Audio/metadata/package requirements depend on the delivery path	Validate media consistency, metadata, artwork, and package structure
Amazon Polly	Character pricing, engine capabilities, quotas, and asynchronous limits differ by engine	Estimate by engine; throttle; retry; record billed characters and model version

Product Requirements Document

Goals and non-goals

- Goal: let a new user create a persistent project, upload a supported manuscript, confirm chapters, generate and approve a sample, understand cost, and start a durable production run.
- Goal: let an editor identify a problem at chapter/segment level, modify direction or text, regenerate only affected audio, rerun QC, and preserve an audit trail.
- Goal: let an organization control users, roles, usage, provider credentials, retention, exports, and portfolio status.
- Goal: produce deterministic, documented deliverables rather than an opaque media file.
- Non-goal: replace professional editorial judgment, guarantee retailer acceptance, or infer that the user owns rights.
- Non-goal: become a consumer audiobook player or a full digital audio workstation.

End-to-end user journey

1. Create an organization or join one; select the applicable plan and project role.
2. Create a project and enter title, author, narrator/voice intent, language, rights declaration, distribution targets, and retention preference.
3. Upload DOCX, EPUB, TXT, or PDF through a signed upload; receive scan and parsing status without blocking the browser.
4. Review detected title/front matter/parts/chapters, confidence warnings, exclusions, word counts, and estimated runtime; correct structure before audio generation.
5. Choose an authorized provider and voice, configure style/rate/pitch where supported, create pronunciation entries, and generate short A/B samples.
6. Approve a voice direction; receive provider-specific character counts, estimated cost, time range, quota impact, and cache savings.
7. Start production; observe durable progress, chapter states, retries, spend, and warnings; safely leave and return later.
8. Review chapter audio alongside text; add anchored issues, edit direction, regenerate selected segments or chapters, and compare versions.
9. Run target-specific mastering and QC; resolve, waive with reason, or accept each blocking issue according to role.
10. Approve release; produce versioned packages, metadata/cover validation, QC report, cost summary, manifest, and signed downloads.

11. Archive or delete the project according to policy; retain only allowed audit and billing evidence.

Functional requirements catalog

ID	Priority	Domain	Requirement	Acceptance summary
FR-001	Must	Identity	The system shall require authentication for all project, job, asset, usage, and administration operations.	Unauthenticated requests receive a stable 401 response; public health excludes sensitive data.
FR-002	Must	Organizations	The system shall place every project inside an organization and every user relationship inside a membership.	No project or asset exists without org_id; cross-org queries are denied and tested.
FR-003	Must	Roles	The system shall support Owner, Admin, Producer, Editor, Reviewer, and Viewer permissions.	Permission matrix is enforced server-side for every mutation and download.
FR-004	Should	Invitations	Admins shall invite, resend, expire, and revoke memberships.	Invite tokens expire; acceptance is audited; duplicate membership is prevented.
FR-005	Must	Projects	Users shall create, list, filter, open, rename, archive, restore, and delete projects according to role.	Lifecycle transitions are explicit; delete obeys retention and legal-hold rules.
FR-006	Must	Metadata	Projects shall store title, subtitle, author, narrator/voice attribution, language, publisher, description, ISBN/identifiers, copyright year, and distribution targets.	Required fields vary by export profile and are validated before release.
FR-007	Must	Rights	Project creation shall capture a rights declaration and synthetic-voice/voice-consent provenance.	Production cannot reach release-approved without the required declarations.
FR-008	Must	Uploads	The system shall issue short-lived signed upload instructions rather than proxying large manuscripts through the API.	Upload completes directly to quarantine storage and cannot be read as trusted until scanned.
FR-009	Must	Validation	Uploads shall be checked for size, extension, detected MIME, archive expansion limits, malware, encryption, and parser support.	Invalid files are quarantined/rejected with a safe reason and internal error code.
FR-010	Must	Versions	Each accepted upload shall create an immutable manuscript version with checksum and provenance.	Replacing a manuscript creates a new version; previous versions remain addressable per policy.
FR-011	Must	Parsing	The system shall extract supported document text and structure asynchronously.	API remains responsive; status and parse warnings are durable.
FR-012	Should	OCR	The system should detect image-only PDFs and offer OCR as a separate bounded operation.	OCR is opt-in when cost/privacy differs; confidence and page warnings are visible.
FR-013	Must	Structure	The parser shall classify title, front matter, parts, chapters, sections, back matter, and excluded content.	User can inspect and override every proposed boundary before production.
FR-014	Must	Confidence	Automated structure decisions shall expose confidence and reasons.	Low-confidence items appear in a review queue; no hidden destructive split occurs.
FR-015	Must	Chapter editor	Users shall rename, reorder, merge, split, include/exclude, and mark chapter type.	All changes create a new structure revision and invalidate affected downstream work.
FR-016	Should	Text editing	Authorized users should make bounded text corrections without overwriting the source manuscript.	Edited production text is versioned and diffable against the source.
FR-017	Must	Normalization	The system shall apply a versioned text-normalization profile before synthesis.	Normalized text is previewable; transformations are traceable and reversible.
FR-018	Must	Segmentation	The system shall segment on semantic boundaries while respecting provider limits.	No ordinary word is cut; segments remain linked to source offsets.

ID	Priority	Domain	Requirement	Acceptance summary
FR-019	Must	Providers	The system shall expose only providers and models authorized and healthy for the organization and deployment.	Unavailable or disallowed providers cannot be selected or invoked.
FR-020	Must	Voices	Users shall search/filter voices by language, locale, capability, gender descriptor where supplied, provider, and model support.	Voice results include availability, terms classification, and supported controls.
FR-021	Must	Samples	Users shall generate short samples before approving a full run.	Samples carry cost, provider/model, voice, text, controls, and expiry metadata.
FR-022	Should	A/B compare	Users should compare two or more samples with the same normalized text.	Playback can switch without losing labels; one candidate can be approved.
FR-023	Must	Voice direction	A project shall store voice, engine/model, locale, rate, pitch, style, and other supported controls.	Unsupported combinations are rejected before enqueue.
FR-024	Should	Chapter voices	Users should override the default voice/direction for a chapter.	Overrides are visible, versioned, and included in estimates.
FR-025	Later	Multi-cast	The system may assign voices to confirmed roles or spans.	Automatic attribution never starts billable synthesis without human confirmation.
FR-026	Must	Pronunciation	Users shall create project and organization pronunciation entries.	Entries support source term, replacement/phoneme, locale, scope, notes, and test sample.
FR-027	Must	SSML safety	The system shall validate generated/user SSML against provider capabilities.	Unsafe/unsupported tags are rejected; rendered canonical SSML is stored for provenance.
FR-028	Must	Estimate	The system shall estimate billable characters, cache reuse, provider cost, output runtime, and broad completion time before a production run.	Estimate records assumptions and expires if relevant inputs change.
FR-029	Must	Budget approval	Full production shall require explicit approval of an estimate or organization policy.	Approved amount, approver, time, and max overage are recorded.
FR-030	Must	Quotas	The system shall enforce plan, org, provider, and user limits before and during jobs.	Jobs pause/fail safely before uncontrolled spend; limits are explainable.
FR-031	Must	Jobs	Production shall run as durable jobs with stable identifiers and persisted state.	API/worker restart does not erase accepted work or completion evidence.
FR-032	Must	Idempotency	Create-job and other costly mutations shall support idempotency keys.	Duplicate client retries return the original outcome without duplicate billing.
FR-033	Must	Attempts	Each segment synthesis shall record attempts, provider request identity where available, latency, classification, billed units, and result checksum.	Operators can distinguish retry, provider failure, invalid input, and internal defect.
FR-034	Must	Retry	Retryable failures shall use exponential backoff with jitter and bounded attempts.	Nonretryable failures do not loop; manual retry creates a new attempt.
FR-035	Must	Cancellation	Authorized users shall cancel queued/running production.	No new provider call starts after cancellation is acknowledged; final state is durable.
FR-036	Should	Pause/resume	Authorized users should pause scheduling and later resume unfinished work.	In-flight calls finish safely; pending segments remain queued but undispatched.
FR-037	Must	Reconciliation	The system shall detect expired leases/orphaned tasks and reconcile them.	No job remains silently running after worker loss beyond the configured threshold.
FR-038	Must	Progress	Users shall see overall, chapter, segment, cost, retry, and QC progress.	Status updates are monotonic where applicable and survive browser refresh.

ID	Priority	Domain	Requirement	Acceptance summary
FR-039	Must	Cache	Reusable audio shall be keyed by canonical text plus the complete versioned synthesis configuration.	Changing any audible control or provider/model version invalidates the key.
FR-040	Must	Cache scope	Cache reuse shall default to organization scope with explicit retention.	Cross-org lookup is impossible unless a separate approved policy is implemented.
FR-041	Must	Selective regeneration	Users shall regenerate selected segments/chapters without rerunning unaffected synthesis.	Invalidation graph identifies exactly which assemblies/QC/packages must be rebuilt.
FR-042	Must	Audio validation	Every provider response shall be verified as nonempty decodable audio with plausible duration.	Invalid responses are not cached as success and enter retry/error handling.
FR-043	Must	Assembly	The system shall assemble segments and chapters according to a versioned timing profile.	Gap durations and fades are explicit, tested, and recorded.
FR-044	Must	Masters	The system shall retain a loss-minimizing chapter master before lossy export where practical.	Exports can be reproduced without repeated lossy transcodes.
FR-045	Must	Mastering	The system shall apply a versioned mastering chain per target profile.	Before/after measurements and tool/profile versions are stored.
FR-046	Must	QC	The system shall measure technical criteria required by the selected export profile.	Each check returns value, unit, threshold, severity, pass/fail, method, and version.
FR-047	Must	Consistency	QC shall compare chapters for loudness, channel, sample-rate, encoding, and pacing anomalies.	Project-level inconsistency can block release even when individual files pass.
FR-048	Should	Content QC	The system should flag suspicious truncation, duplicate audio, extended silence, clicks, or text/audio duration anomalies.	Flags are advisory unless a profile explicitly makes them blocking.
FR-049	Must	Review	Reviewers shall play chapter/segment audio alongside the production text.	Playback position can create an anchored issue with text/audio context.
FR-050	Must	Issues	Users shall create, assign, comment on, resolve, reopen, and waive QC/editorial issues.	Every transition is audited; waivers require reason and eligible role.
FR-051	Must	Approvals	The workflow shall support sample, chapter/QC, and release approvals.	Approval binds to exact artifact/config versions and becomes stale after invalidating changes.
FR-052	Should	Comparison	Reviewers should compare audio versions for a regenerated segment.	The UI labels versions and preserves issue context.
FR-053	Must	Exports	Users shall generate MP3 chapter sets, merged MP3, M4B, and supporting manifests according to entitlements.	Each package is immutable, checksummed, and linked to its target profile.
FR-054	Must	Metadata	Exports shall include validated bibliographic and audio metadata supported by the format.	Missing/invalid target fields block release or produce an explicit waiver.
FR-055	Must	Cover	Users shall upload and validate cover artwork for dimensions, aspect, format, and file size.	Original and derived artwork versions are stored and traceable.
FR-056	Should	Credits/sample	The system should guide opening credits, closing credits, and retail sample creation.	Target profile validates presence, duration, and placement when applicable.
FR-057	Must	Manifest	Every release package shall include a human-readable and machine-readable manifest.	Manifest lists files, checksums, durations, encodings, metadata, profile, QC, approvals, and provenance.
FR-058	Must	Downloads	Assets shall be delivered through short-lived signed URLs with authorization and audit.	Raw storage paths and permanent public links are never exposed.
FR-059	Should	Notifications	Users should receive in-app and email/webhook notifications for milestones and action-required states.	Preferences and delivery outcomes are recorded; no manuscript text enters notifications.

ID	Priority	Domain	Requirement	Acceptance summary
FR-060	Should	Webhooks	Organizations should configure signed, retryable webhooks for project/job events.	Payloads are versioned; signatures, retry, replay, and disable controls exist.
FR-061	Must	Usage ledger	The system shall record estimated and actual billable units, provider cost, platform units, and credits by project/job.	Ledger is append-only and reconcilable to provider invoices where possible.
FR-062	Should	Billing	Plan entitlements and usage charges should integrate with the billing provider without making the billing provider the system of record for production.	Production policy can be evaluated locally even during billing outage.
FR-063	Must	Retention	Organizations shall have documented retention defaults for uploads, intermediates, cache, outputs, and logs.	Lifecycle deletion is observable and honors active holds/contract rules.
FR-064	Must	Audit	Security- and release-relevant actions shall emit immutable audit events.	Events include actor, org, object, action, time, request/correlation ID, and safe change metadata.
FR-065	Must	Admin	Authorized operators shall inspect jobs, attempts, provider health, queues, quotas, and storage without reading manuscripts by default.	Support access is least-privilege, time-bound where possible, and audited.
FR-066	Should	Templates	Organizations should save production templates for voice direction, normalization, timing, mastering, export, and review gates.	Templates are versioned; updating one does not silently alter existing projects.
FR-067	Should	Portfolio	Organizations should view project stage, owner, risk, estimate/actual cost, QC state, and deadlines.	Portfolio view is filterable and does not load full manuscript/audio content.
FR-068	Must	Errors	The system shall use stable, user-safe error codes and preserve detailed internal diagnostics.	UI provides next action; secrets/provider payloads are not leaked.
FR-069	Must	Accessibility	Core workflows shall conform to WCAG 2.2 AA.	Automated and manual keyboard, focus, name/role/value, contrast, and live-region tests pass.
FR-070	Should	Localization	The UI should be prepared for localized strings, dates, currencies, and right-to-left layouts.	No critical UI text is hardcoded inside reusable components.

Non-functional requirements catalog

ID	Category	Measurable requirement
NFR-001	Availability	Metadata/API control plane monthly availability ≥99.9% after production launch.
NFR-002	Durability	Accepted project/job state is committed before success response; no silent loss on process restart.
NFR-003	Recovery	Initial production targets: database RPO ≤5 minutes and service RTO ≤60 minutes, tested at least twice yearly.
NFR-004	API latency	p95 ≤300 ms for ordinary metadata reads and ≤500 ms for ordinary writes, excluding media transfer and external-provider work.
NFR-005	Job acknowledgement	A valid start request returns an accepted job identity within 2 seconds without waiting for synthesis.
NFR-006	Progress freshness	Running job progress is visible within 5 seconds of a persisted state change under normal operation.
NFR-007	Scalability	Worker concurrency scales independently from API instances; provider and org limits remain enforced across replicas.
NFR-008	Backpressure	The system rejects or queues safely when quotas/capacity are exhausted; it does not spawn unbounded work.
NFR-009	Idempotency	Costly create operations are repeat-safe for at least 24 hours using organization-scoped idempotency keys.

ID	Category	Measurable requirement
NFR-010	Consistency	Project and approval mutations use optimistic concurrency/version checks to prevent lost updates.
NFR-011	Encryption	TLS in transit; managed encryption at rest for database, object storage, backups, and queue persistence.
NFR-012	Secrets	Provider and platform credentials are stored in a secret manager and never logged or returned to the browser.
NFR-013	Tenant isolation	Authorization tests cover every object path; cache keys and storage prefixes cannot bypass org scoping.
NFR-014	Upload safety	Parsers run with bounded CPU, memory, wall time, file count, and decompressed-size limits.
NFR-015	Privacy	Manuscript/audio contents are excluded from application logs, analytics, traces, and error payloads by default.
NFR-016	Auditability	Audit events and release manifests are append-only with tamper-evident storage/controls appropriate to plan.
NFR-017	Observability	Every request and job carries correlation IDs; metrics cover queue, provider, cost, QC, storage, and errors.
NFR-018	Accessibility	WCAG 2.2 AA for core create, review, approve, and export flows.
NFR-019	Browser support	Current and previous major versions of Chrome, Edge, Firefox, and Safari; responsive at 360px and above.
NFR-020	Reproducibility	A release artifact can be rebuilt from immutable source/config versions and documented toolchain versions.
NFR-021	Portability	Provider adapters and object storage are behind interfaces; no provider-specific identifier becomes the project primary key.
NFR-022	Compatibility	Version API responses and webhook payloads; preserve documented compatibility or publish migration windows.
NFR-023	Maintainability	Python type checking, linting, formatting, and tests; TypeScript strict mode, linting, formatting, and tests all gate merge.
NFR-024	Supply chain	Lockfiles, dependency review, secret scanning, SAST, container scanning, and SBOM generation gate release.
NFR-025	Data lifecycle	Deletion jobs are idempotent, observable, retryable, and produce evidence without retaining deleted content.
NFR-026	Cost control	No paid provider call occurs without an eligible org, available budget, rate-limit token, and usage reservation.
NFR-027	Provider resilience	Circuit breakers/degradation protect queues; retry storms cannot multiply provider cost.
NFR-028	Audio determinism	Profile version and tool versions are recorded; golden fixtures detect meaningful processing drift.
NFR-029	Release safety	Database migrations are backward-compatible during deployment; rollback does not corrupt active jobs.
NFR-030	Supportability	Operator tooling traces a user-visible job end to end without shell access.

Functional Specification

Information architecture

Surface	Primary contents
Home / portfolio	Recent projects, action required, running jobs, spend/usage, deadlines, invitations
Project overview	Stage, source version, production configuration, estimate/actuals, approvals, release history
Manuscript	Upload versions, parse status, structure editor, chapter inclusion, warnings, production text diff
Voice lab	Provider/model eligibility, voice search, sample text, A/B samples, direction, approval
Pronunciations	Organization and project lexicons, locale, test audio, conflicts, import/export
Production	Job controls, chapter/segment progress, attempts, cache reuse, spend, action-required errors
Review	Text-synchronized playback, issues, comments, assignments, version compare, regeneration
Quality	Target profile, measurements, consistency view, blockers, waivers, approval
Exports	Cover/metadata, package profiles, manifests, signed downloads, release history
Organization	Members, roles, providers/BYOK, quotas, templates, billing, retention, webhooks, audit
Operations	Provider health, queues, jobs, attempts, alerts, storage, reprocessing, support access

Project stage model

Stage	Meaning	Exit trigger	Next
Draft	Project exists; metadata/source may be incomplete	Upload or edit metadata	Ingesting
Ingesting	Upload is quarantined/scanned/parsed	System pipeline	Needs structure review / Ingestion failed
Needs structure review	Chapters or warnings require confirmation	Producer/editor confirms structure	Ready for direction
Ready for direction	Production text is valid; voice/sample not approved	Configure and approve sample	Ready to estimate
Ready to estimate	Synthesis configuration approved	Generate/approve current estimate	Ready for production
Ready for production	Estimate and budget approval are current	Start durable job	Producing
Producing	One or more production jobs active	System/user pause/cancel/error	Needs attention / Quality review
Needs attention	Blocking failure or user decision required	Retry, edit, change provider, approve overage	Producing / Ready for production

Stage	Meaning	Exit trigger	Next
Quality review	Required audio exists; QC/review incomplete	Resolve/waive issues; approve release	Ready to export
Ready to export	Release approval is current	Generate package	Released
Released	Immutable package and manifest exist	New source/config creates new release line	Released / Draft revision
Archived	Project hidden from active work; retention continues	Restore or delete per policy	Prior eligible stage / Deleted

Job and attempt state model

Project stage describes customer workflow. Job state describes durable execution. Segment attempts describe individual provider calls. These concepts must not be collapsed into one status string.

State	Semantics	Permitted action
Queued	Accepted and eligible; waiting for capacity	Cancel; priority update by policy
Running	Worker owns a renewable lease and dispatches eligible units	Pause; cancel; automatic retry
Pausing	No new units dispatched; in-flight calls finish	Becomes Paused
Paused	Durable unfinished work retained	Resume or cancel
Cancelling	Cancellation acknowledged; cleanup/reconciliation underway	Becomes Cancelled
Retry scheduled	Retryable failure waiting for backoff	Automatically returns to Running
Needs input	Cannot proceed without user decision	User supplies correction/approval then resumes
Succeeded	All required artifacts and accounting committed	Terminal
Failed	Nonretryable or attempt budget exhausted	Manual retry creates new job/revision
Cancelled	No further provider dispatch; partial artifacts retained per policy	Terminal

Detailed screen behavior

Portfolio

- Default to action-required projects rather than a vanity total.
- Filter by stage, owner, target, language, provider, deadline, QC state, and archived status.
- Show estimates and actuals only to eligible roles; avoid exposing content snippets.
- Running cards display persisted progress and last update; stale jobs are explicitly labeled.

Create project

- Use a short first step: title, language, ownership/rights acknowledgment, target, and file.
- Save a draft immediately after the first valid step and allow exit/re-entry.
- Explain how synthetic narration eligibility differs by target; never imply universal distribution.
- Do not request provider credentials inside the project wizard; route admins to organization settings.

Structure editor

- Use a left chapter outline and right text/metadata pane with clear confidence warnings.
- Allow drag reorder, split at cursor, merge with prior/next, include/exclude, type change, and title edit.
- Show word/character count, estimated duration, and downstream invalidation before committing changes.
- Preserve the immutable source and show production-text differences.

Voice lab

- Voice selection is a decision workspace, not a giant unfiltered voice grid.
- Use fixed sample text when comparing candidates; include user-selected representative passages.
- Display provider/model, locale, allowed controls, price class, policy classification, and sample cost.
- Approval binds the exact voice-direction version and expires after relevant change.

Production

- Show overall progress, chapter progress, queued/running/retry/failed counts, estimate versus actual cost, and last event.
- Provide pause/cancel/retry only when semantically valid; require confirmation for cost-bearing retry.
- A failure card includes what happened, whether work is safe, what was already charged, and the next safe action.
- Leaving the page has no effect on execution.

Review

- Synchronize current playback with chapter/segment text when timing data permits.
- Create issues at playback time, source span, segment, or chapter; label type and severity.
- Regeneration preview lists invalidated assets, estimated incremental cost, and approval staleness.
- Comparisons preserve the old version until the replacement is approved.

Quality

- Separate technical pass/fail from editorial warnings and policy eligibility.

- Show measured value, target, method, affected file, severity, and remediation.
- Block release only according to the selected profile; waivers require reason and eligible role.
- Project consistency view highlights chapter outliers even when each file individually passes.

Exports

- Target profile drives required files, metadata, cover rules, credits/sample, and naming.
- Package generation creates a new immutable release candidate, not an overwrite.
- Downloads are short-lived; manifest and QC report remain linked to the package.
- Changing a released project creates a new release line with comparison to the prior release.

Role and permission matrix

Permission	Owner	Admin	Producer	Editor	Reviewer	Viewer
View projects/audio	Yes	Yes	Yes	Yes	Yes	Yes
Create/edit project	Yes	Yes	Yes	Limited	No	No
Upload/structure manuscript	Yes	Yes	Yes	Yes	No	No
Configure provider/voice	Yes	Yes	Yes	Limited	No	No
Approve estimate/start paid work	Yes	Policy	Policy	No	No	No
Edit production text/pronunciations	Yes	Yes	Yes	Yes	Comment	No
Create/resolve issues	Yes	Yes	Yes	Yes	Yes	No
Approve QC/release	Yes	Policy	Policy	No	Policy	No
Download release assets	Yes	Yes	Yes	Policy	Policy	Policy
Manage members/providers/billing	Yes	Yes	No	No	No	No
View cost and audit	Yes	Yes	Policy	No	No	No
Delete/restore project	Yes	Policy	No	No	No	No

Policy means organization configuration may grant the action, but the API still evaluates role, project assignment, plan entitlement, approval threshold, and object state. The frontend may hide unavailable actions; it is never the enforcement layer.

Error and recovery behavior

Failure	System behavior	User behavior	Invariant
Invalid upload	Reject/quarantine before trusted parsing	Explain supported formats and safe retry	No project loss
Parser timeout	Mark ingestion attempt failed; retain quarantined source by policy	Retry or choose alternate extraction/OCR	No duplicate manuscript version

Failure	System behavior	User behavior	Invariant
Provider throttling	Backoff with jitter; preserve queued work	Show delayed, not failed, until budget exhausted	No retry storm
Provider model/voice unavailable	Stop affected units; do not silently substitute	Choose replacement and generate comparison sample	Approval becomes stale
Budget exceeded	Pause before unreserved paid calls	Approve overage, change provider, reduce scope, or cancel	Actual charges remain visible
Worker loss	Lease expires; reconciliation requeues safe units	No customer action unless repeated	Idempotency prevents duplicate artifacts/charges where possible
Audio decode/QC failure	Do not mark segment/chapter successful	Retry synthesis or inspect provider/audio worker error	Bad bytes never enter reusable cache
Signed link expired	Asset remains private	Request a new authorized link	No permanent public URL
Concurrent edit	Reject stale mutation with conflict payload	Reload/compare and reapply	No silent overwrite
Notification failure	Production state remains authoritative	Retry notification; surface delivery status to admins	No job rollback

Technical Specification

Architecture decision: modular monolith with worker planes

The recommended target is one product codebase with explicit domain modules and independently scalable process types: web/API, ingestion worker, synthesis worker, and audio/QC worker. PostgreSQL is the source of truth. Object storage holds large immutable artifacts. Redis or a managed queue coordinates work and short-lived locks. This creates production-grade boundaries without the deployment and consistency burden of many microservices.

> **Evolution rule:** Extract a service only when a module has a distinct scaling, security, reliability, or ownership requirement demonstrated by data. Do not split merely because the architecture diagram has boxes.

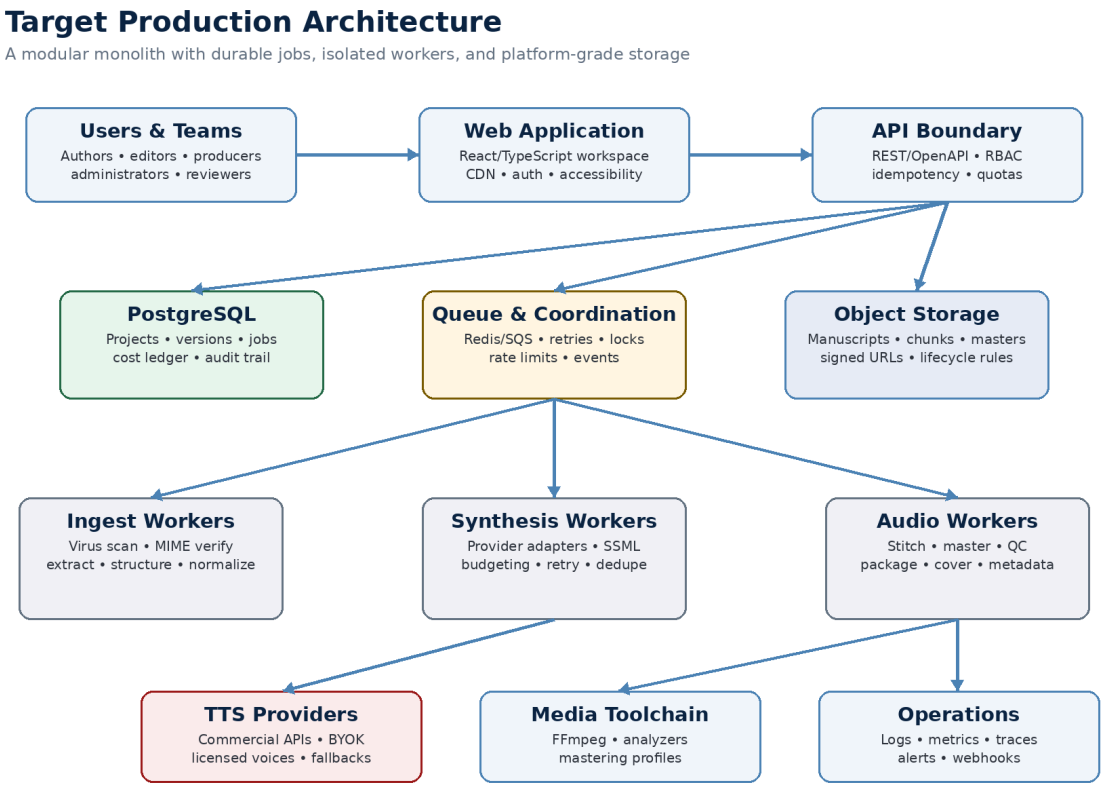

Figure 1. Recommended production architecture and responsibility boundaries.

Processing pipeline

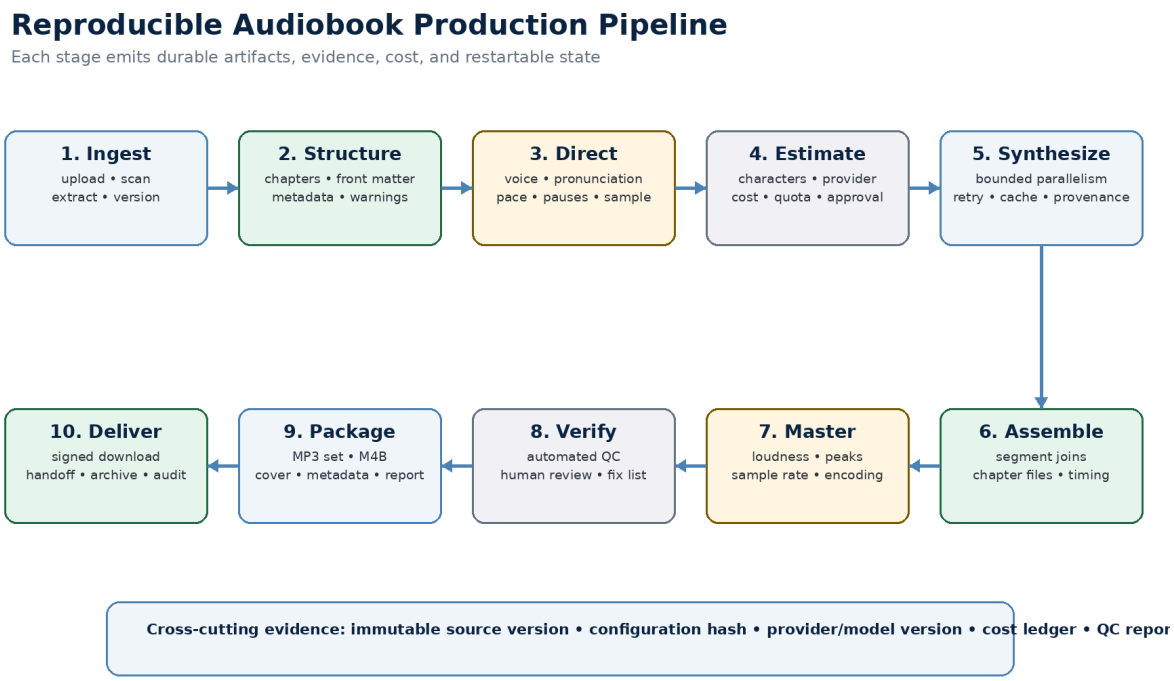

Figure 2. Durable, evidence-producing audiobook pipeline.

Every pipeline stage consumes immutable or versioned inputs and writes a durable result before advancing state. A customer-visible success is returned only after the database references the resulting artifact. Provider calls and media processing never rely on browser presence or API process memory.

Recommended stack

Layer	Recommended default	Rationale
Web	React + TypeScript + Vite initially; React Router, TanStack Query, Zod, accessible component primitives	Avoid an unnecessary framework rewrite; add routing, server-state discipline, and runtime validation
API	Python modular API; retain Flask during foundation or migrate to FastAPI after service extraction	Typed schemas/OpenAPI matter more than framework branding
Domain/data	SQLAlchemy 2 + Alembic + PostgreSQL	Transactions, migrations, relational integrity, JSON only for bounded provider/profile data
Jobs	Celery/Dramatiq with Redis or managed queue; leases and reconciliation in DB	Durable attempts, backoff, priorities, cancellation, independent worker scaling
Objects	S3-compatible object storage with quarantine/trusted prefixes, KMS, lifecycle, signed URLs	Audio/manuscripts do not belong on an API container filesystem
Media	Pinned FFmpeg/ffprobe, loss-minimizing intermediates, versioned filters/profiles	Reproducible assembly, mastering, measurement, and packaging

Layer	Recommended default	Rationale
Auth	Managed OIDC provider or vetted library; organization/role policy in application DB	Do not build password security casually; keep business authorization local
Observability	OpenTelemetry, structured JSON logs, metrics, traces, error tracking, provider/cost dashboards	Cross-process correlation and safe diagnostics
Infrastructure	Managed Postgres/object store/queue where possible; Terraform/OpenTofu; containers; CDN	Reduce undifferentiated operations while preserving portability

Module boundaries

Module	Owns	Boundary
identity	Users, organizations, memberships, role evaluation, support access	Does not contain billing or project content
projects	Project lifecycle, metadata, targets, rights declarations, stage	Does not own binary files
ingestion	Upload sessions, quarantine, scan, extraction, structure proposals, manuscript versions	Does not call TTS
editorial	Production text revisions, chapters, segments, pronunciation sets, voice direction	Produces versioned instructions
providers	Adapter contracts, capabilities, credentials references, health, policy classification	No project workflow state
estimation	Billable units, cache prediction, cost/time estimate, reservations, approvals	No direct provider call
production	Jobs, task graph, leases, attempts, retry/cancel/pause/reconcile, progress	Orchestrates but does not implement vendor/media details
artifacts	Object metadata, checksums, lineage, signed access, retention, cache lookup	Binary data stays in object storage
audio	Decode validation, joins, masters, mastering, analysis, package formats	No user authorization logic
quality	Profile checks, results, consistency, issue linkage, waivers, approvals	Profile versions immutable
releases	Release candidates, manifests, downloads, target eligibility, release history	Consumes approved artifacts
usage	Reservations, actual units, provider cost, platform metering, credits	Append-only financial evidence
notifications	In-app/email/webhook delivery and retry	Consumes events; never drives production truth
audit	Immutable security/release events and exports	No manuscript/audio body content

Core data model

Entity	Key fields	Purpose
Organization	id, name, plan, policy, retention, status	Tenant/security boundary
User / Membership	user_id, org_id, role, state, invited_by	Identity-to-tenant relationship
Project	org_id, stage, metadata, target set, owner, version	Persistent production workspace
RightsDeclaration	project_id, scope, synthetic disclosure, voice consent refs, accepted_by/time	Eligibility and audit evidence
ManuscriptVersion	project_id, object_id, checksum, mime, source, parse state	Immutable uploaded source
StructureRevision	manuscript_version_id, revision, proposal/confirmed_by	Title/front matter/parts/chapters map
ChapterRevision	structure_revision_id, position, type, title, text checksum	Versioned production unit
Segment	chapter_revision_id, source offsets, canonical text, checksum	Smallest synthesis/review unit
VoiceDirectionVersion	project/chapter scope, provider/model/voice, controls, locale	Audible synthesis configuration
PronunciationSetVersion	org/project scope, entries, locale, checksum	Versioned lexicon applied to segments
Estimate	input/config hashes, billable units, reuse, price snapshot, expiry	Pre-spend evidence
BudgetApproval	estimate_id, approved_by/time, amount, overage	Authority for paid work
Job	project_id, type, state, priority, requested_by, idempotency key	Durable orchestration
JobUnit	job_id, segment/chapter, state, lease, dependency count	Restartable task graph node
Attempt	unit_id, provider request ref, state, timings, error class, billed units	One external/internal execution
Artifact	org_id, kind, object key, checksum, media info, lineage hash	Metadata for immutable binary result
CacheEntry	scope, synthesis hash, artifact_id, expiry/status	Safe reusable artifact pointer
QCProfileVersion	target, rules, severities, analyzer versions	Immutable interpretation of quality
QCResult	artifact/profile, measurements, pass state, analyzer evidence	Objective file evidence
Issue / Comment	project, anchor, type, severity, assignee, state	Editorial/technical remediation
Approval	scope, artifact/config hashes, role, actor, state/time	Human decision bound to exact version
Release	project, target/profile, state, package artifact, prior release	Immutable deliverable line
UsageEntry	org/project/job/attempt, quantity, unit, provider cost, currency	Append-only unit economics
AuditEvent	actor/org/object/action/request/time/safe delta	Security and release history

Relational and concurrency rules

- All tenant-owned tables include org_id directly or have an enforced, indexed path to it; authorization filters are applied before object lookup is returned.
- Project has an optimistic version column. Mutations carry If-Match/version and fail with a conflict when stale.
- Source, configuration, profile, and released artifacts are immutable. A change creates a new version and lineage edge.
- Job state transitions use compare-and-set semantics. Workers update only units for which they hold a valid lease.
- Usage entries and audit events are append-only. Corrections are compensating entries/events, not destructive edits.
- Artifact records become available only after checksum/media validation and object commit; partial uploads remain quarantined.
- Approvals reference configuration and artifact hashes. Any invalidating change marks the approval stale rather than deleting history.

API surface

Method	Path	Purpose	Critical semantics
GET/POST	/v1/projects	List/create projects	Auth + org membership; POST idempotent
GET/PATCH/DELETE	/v1/projects/{projectId}	Read/update/archive/delete	ETag/version; role/state checks
POST	/v1/projects/{id}/upload-sessions	Create signed manuscript upload	Type/size policy; idempotent
POST	/v1/upload-sessions/{id}/complete	Commit upload to quarantine	Checksum/size verification
GET	/v1/projects/{id}/manuscripts	List source versions and ingestion state	Paged; no raw object key
POST	/v1/manuscripts/{id}/ingestion-jobs	Parse/structure a version	Durable job
GET/PATCH	/v1/projects/{id}/structure	Read/confirm/edit structure revision	Optimistic concurrency
GET/PATCH	/v1/chapters/{id}	Read/update production chapter	Creates new revision
GET	/v1/voices	Search eligible voices/capabilities	Provider health/policy filtered
POST	/v1/projects/{id}/samples	Generate sample job	Estimate/usage rules; idempotent
POST	/v1/projects/{id}/voice-direction-versions	Save direction	Schema/capability validation
GET/POST/PATCH	/v1/projects/{id}/pronunciations	Manage project lexicon	Versioned; testable
POST	/v1/projects/{id}/estimates	Create current production estimate	Snapshot prices/config/cache
POST	/v1/estimates/{id}/approvals	Approve budget	Eligible role/policy

Method	Path	Purpose	Critical semantics
POST	/v1/projects/{id}/production-jobs	Start durable production	Idempotency + current approval
GET	/v1/jobs/{jobId}	Read progress/state/cost	Org/project authorization
POST	/v1/jobs/{jobId}:pause|resume|cancel|retry	Control execution	State-machine validation
POST	/v1/segments/{id}/regeneration-jobs	Selective regeneration	Impact/estimate confirmation
GET	/v1/projects/{id}/artifacts	List authorized assets and lineage	Metadata only
POST	/v1/artifacts/{id}/download-sessions	Create short signed download	Role, retention, audit
POST	/v1/projects/{id}/qc-runs	Run selected profile	Durable analysis job
GET	/v1/projects/{id}/qc-results	Read measurements/blockers	Filter by release candidate
GET/POST/PATCH	/v1/projects/{id}/issues	Review issue workflow	Anchors + audit
POST	/v1/projects/{id}/approvals	Sample/QC/release approval	Binds exact version hashes
POST	/v1/projects/{id}/releases	Create target release candidate	Eligibility + approval gate
GET	/v1/releases/{id}/manifest	Read human/machine manifest	Immutable
GET	/v1/usage	Usage/cost summaries	Role-limited; paged
GET/PATCH	/v1/organizations/{id}/settings	Policy, provider, quota, retention	Admin/owner only
GET	/v1/organizations/{id}/audit-events	Audit search/export	Admin/owner; content-safe
POST	/v1/organizations/{id}/webhooks	Configure signed webhooks	Secret shown once

> **API contract:** Publish OpenAPI 3.1, generate client types where practical, validate requests and responses at boundaries, and use a stable error envelope containing code, message, correlation_id, retryable, and field_errors. Never return provider secrets, raw stack traces, or object keys.

Job orchestration design

1. API validates authorization, project state, current estimate/approval, idempotency key, quotas, and usage reservation inside a database transaction.
2. API creates Job and JobUnit records, commits them, then publishes the job identity to the queue using an outbox pattern.
3. Worker claims eligible units with a lease and heartbeat. Lease ownership is checked on progress/finalization writes.
4. Worker loads immutable input/config versions, obtains provider rate-limit tokens, and creates an Attempt before external work.
5. Provider response is written to temporary object storage, decoded and measured, checksummed, promoted to trusted storage, then linked by Artifact.
6. Usage actuals and attempt outcome are committed with the unit transition. Difference from reservation is released or charged according to policy.

7. Dependent units become eligible only when all required upstream artifact references are committed.
8. Terminal job outcome is derived from durable unit states and required outputs, not a Python thread finishing.
9. Reconciler finds expired leases, stuck transitions, orphan objects, unreleased reservations, and outbox failures; repairs idempotently or alerts.

Provider adapter contract

Operation	Contract	Implementation rule
capabilities()	Voices, models/engines, locales, SSML tags, controls, limits, formats, pricing class	Cached with short TTL and provider health
health()	Availability, latency, auth validity, quota/degradation	Bounded; never on every user request
estimate(input)	Billable units and price snapshot	Uses canonical provider billing rules
synthesize(request)	Audio bytes/reference plus provider metadata	Idempotency where provider supports; timeout and cancellation policy
normalize_error(error)	Retryable/throttled/auth/invalid/policy/provider/internal	No raw vendor payload to user
provenance()	Provider, model/engine, voice ID/version, request settings, region, terms class	Stored with attempt/artifact

The current EdgeProvider and PollyProvider are useful prototypes for this pattern. The contract must expand from four methods to capability, policy, estimate, provenance, normalized error, and health semantics. A provider should never be substituted automatically after sample approval because provider/model changes are audible and may change rights or distribution eligibility.

Cache and invalidation design

The existing SHA-256 cache is the right instinct but an incomplete key. The production synthesis hash should be calculated from a canonical payload and stored with readable provenance.

Hash component	Required content
Tenant scope	organization_id or approved project scope
Canonical text	normalized segment text or canonical SSML, including pronunciation resolution
Provider identity	provider, region/endpoint class, model/engine, voice identifier and version where available
Audible controls	rate, pitch, style, volume, language/locale, sample rate, output codec settings
Pipeline versions	normalizer, segmenter, SSML renderer, adapter, mastering pre-profile if applicable

Hash component	Required content
Policy	commercial/terms classification and retention eligibility

- Cache success only after decode and plausibility checks.
- Reference-count or lineage-track artifacts; never delete an artifact still used by an approved/released package.
- An edit invalidates the changed segment synthesis and all downstream chapter/book assemblies, QC results, approvals, and releases - not unrelated segments.
- Store a reasoned invalidation event so the UI can explain why an approval or export became stale.
- Do not expose global cache statistics to ordinary users; show only their project/org savings.

Ingestion and text pipeline

Stage	Required behavior
Quarantine	Signed upload, checksum, scan, MIME/extension agreement, archive and parser budgets
Extract	DOCX styles/lists/tables/footnotes where supported; EPUB spine/TOC; TXT encoding; PDF text/OCR path
Classify	Document title, metadata, front matter, parts, chapters, sections, notes, back matter, exclusions
Normalize	Whitespace, typography, abbreviations, dates, numbers, URLs, citations, symbols, repeated headers/footers
Confirm	Confidence/reason display and user structure edits
Segment	Paragraph/sentence/word-aware provider-safe spans with source offsets
Render direction	Pronunciation, pauses, emphasis, language spans, provider-compatible SSML
Version	Checksums and immutable links at each stage

> **Editorial safety:** Normalization must be inspectable. Never silently change names, acronyms, numbers, legal citations, URLs, or dialogue punctuation because a rule 'usually sounds better.' Risky transformations require a preview or explicit profile.

Audio mastering and quality

The current qc_check is a useful smoke test, but it should not be labeled compliance. The production system needs analyzers and mastering profiles that are target-specific and versioned. Each measured rule stores both the value and method so future analyzer changes do not rewrite history.

Check family	What is measured	Default severity
Decode/container	Codec/container readable; duration and stream count plausible	Blocking
Sample rate/channels	Match target and remain consistent across the release	Blocking
Bitrate/mode	Target bitrate and CBR/VBR requirements	Blocking
Average loudness	Target RMS and/or integrated LUFS as defined by profile	Blocking
Peak	Sample and/or true peak ceiling as defined by profile	Blocking
Noise floor	Measured on appropriate silent/room-tone windows	Blocking/advisory by source type
Head/tail	Required room tone/silence bounds	Blocking
Duration/file size	Target maximum and plausible text-to-duration ratio	Blocking/advisory
Long silence	Unexpected internal silence ranges	Advisory or blocking threshold
Clipping/corruption	Clipped samples, truncated frames, decode errors	Blocking
Chapter consistency	Outlier loudness, channels, sample rate, bitrate, spectral/timing anomalies	Blocking/advisory
Content heuristics	Duplicate segment, suspicious truncation, empty/near-empty, repeated audio	Advisory

For ACX-oriented exports, the profile should be initialized from the current official submission requirements and version-stamped. Because platform rules and synthetic-narration policies can change, the application should display the profile's verified-on date and require policy review before making a public compatibility claim.

Frontend architecture

- Use route-level modules for portfolio, project overview, manuscript, voice lab, production, review, quality, exports, and organization settings.
- Use TanStack Query (or equivalent) for server state, retries, invalidation, background refresh, and optimistic updates; do not duplicate durable job truth in ad hoc component state.
- Generate or share TypeScript API types from OpenAPI and validate uncertain external payloads with Zod.
- Use resilient polling as the first status transport, with visibility-aware intervals, exponential error backoff, and last-known state; add SSE when event volume justifies it.
- Use a real notification/toast system and error boundary. Browser alert() and console-only failure are not production UX.
- Preserve playback URLs only while needed; revoke object URLs and handle expired signed URLs.

- Design keyboard-first review controls, visible focus, accessible labels, live progress regions, reduced-motion behavior, and transcript-equivalent text navigation.
- Feature flags guard incomplete workflows; flags are evaluated server-side for entitlements and client-side for presentation only.

Target repository structure

Path	Contents	Purpose
.github/workflows/	ci.yml, security.yml, release.yml	Recognized automation and protected checks
apps/web/	React/Vite application, route modules, UI tests	Customer workspace
apps/api/	API app, schemas, auth boundary, controllers	Control plane
workers/ingest/	Scan/extract/structure tasks	Untrusted document boundary
workers/synthesis/	Provider dispatch and artifact validation	External TTS boundary
workers/audio/	Assembly, mastering, QC, packaging	Media processing boundary
packages/domain/	Entities, services, policies, state transitions	Framework-light business logic
packages/providers/	Provider interfaces/adapters/contract fixtures	Vendor isolation
packages/profiles/	Normalization, mastering, QC, export profile definitions	Versioned production policy
packages/observability/	Logging, tracing, metrics, error helpers	Consistent telemetry
tests/fixtures/	Documents, canonical text, mock provider, golden audio	Reproducible integration tests
infra/	Terraform/OpenTofu, deployment manifests, dashboards	Environment definition
docs/	ADRs, runbooks, API, threat model, data map	Operational memory
scripts/	Development and release utilities only	No production business logic

Current file to target responsibility map

Current source	Current responsibility	Target responsibility
backend/app.py	Routes + orchestration + global state	Thin API controllers plus production/job domain services; no thread work
services/file_manager.py	Local upload/read/delete	UploadSession, quarantine scanner, parser workers, ArtifactStore

Current source	Current responsibility	Target responsibility
services/text_processor.py	Chapter heuristics, cleanup, chunking	Versioned ingest/structure/normalization/segmentation modules
services/synthesis_cache.py	Local global hash-to-MP3	Tenant-scoped CacheEntry metadata + object storage + lineage/invalidation
services/providers/*	Good provider abstraction prototype	Expanded capability/estimate/policy/provenance/error contract
services/polly_service.py	Older direct AWS service abstraction	Remove or consolidate into one Polly adapter; avoid duplicate provider paths
utils/audio_utils.py	Pydub/FFmpeg merge, simple QC, M4B	Audio worker pipelines, mastering/QC/export profiles, golden fixtures
frontend/src/App.tsx	Single transient workflow	Route shell and persistent project workspace
frontend/src/services/api.ts	Hardcoded Axios singleton	Environment-validated client, generated types, auth, stable errors
ProgressTracker.tsx	Useful happy-path progress	Durable job timeline with retry/cost/stale/action-required states
ci-cd/github-actions.yml	Valid workflow content in ignored location	.github/workflows/ci.yml plus quality/security/release gates

Deployment topology and environments

Environment	Composition	Rule
Local	Compose/dev containers, mock provider, fixture object store, seeded DB	One command, no production credentials
CI	Ephemeral Postgres/Redis/object store, mocked external providers, media fixtures	Deterministic tests and image builds
Preview	Per-branch web/API where cost-safe; mock provider only by default	Review UI/API without paid synthesis
Staging	Production-like managed services, sandbox provider credentials, synthetic test books	Migrations, smoke, load, recovery, release rehearsal
Production	Multi-AZ managed DB/queue/object store, autoscaled workers, CDN, secrets/KMS	Customer workloads and audited operations

- API and worker images are immutable and run as non-root with a read-only root filesystem where practical.

- Readiness verifies critical local dependencies/configuration, not all external providers; provider health has separate degradation status.
- Deployments use backward-compatible migrations, rolling replacement, worker drain, and active-job reconciliation.
- Object storage lifecycle transitions intermediates and expires quarantine/signed URLs without deleting released dependencies.
- Production provider credentials are never mounted into the frontend or committed environment files.

Security, Privacy, Rights, and Governance

Threat model priorities

Threat	Example	Required controls	Risk
Cross-tenant access	IDOR/object key guessing, missing org filter, signed link reuse	Server-side org authorization; opaque IDs insufficient; signed URL TTL/audience; exhaustive tests	Critical
Malicious document	Parser exploit, zip bomb, huge PDF, embedded object, malware	Quarantine, MIME sniffing, scan, sandbox/resource budgets, patched parsers, reject encryption	Critical
Cost abuse	Anonymous/bot traffic, replay, retry storm, compromised account	Auth, quotas, reservation ledger, rate limits, idempotency, anomaly alerts, circuit breakers	Critical
Content leakage	Logs/traces/errors/backups/support access/cache	Content-safe telemetry, encryption, scoped cache, redaction, access audit, retention	Critical
Credential exposure	Committed .env, browser bundle, logs, support export	Secret manager, short-lived roles, rotation, scanning, no client exposure	Critical
Command/media injection	Unsafe FFmpeg args, concat/metadata escaping, crafted filenames	No shell interpolation; fixed argument arrays; safe temp names; sandbox and allowlisted filters	High
Supply-chain compromise	Unpinned packages/images/actions	Lockfiles, hashes where feasible, trusted publishers, SBOM, SCA, image signing, action SHA policy	High
Approval bypass	Stale approval after edit, client-only gate, role escalation	Hash-bound approvals, server state machine, audit, privilege tests	High
Voice/rights misuse	Unauthorized cloning or incompatible distribution	Approved provider catalog, consent references, declarations, policy engine, immutable provenance	High
Deletion failure	Orphaned objects/cache/backups exceed policy	Lifecycle index, idempotent deletion jobs, evidence, backup expiry, legal-hold checks	High

Security control baseline

- Use OWASP ASVS 5.0.0 as the application security verification baseline; choose and document the target assurance level.
- Centralize authorization decisions and test them with a denied-by-default policy matrix.
- Use secure, HttpOnly, SameSite cookies for browser sessions or a vetted OIDC flow; protect state-changing browser requests against CSRF.
- Apply request schema validation, output encoding, CSP, HSTS, secure headers, rate limits, and origin allowlists.
- Encrypt sensitive data at rest and in transit; scope cloud IAM per process; prefer short-lived workload identity over static keys.

- Record admin/support access and sensitive exports. Do not let operators browse manuscript content by default.
- Run SAST, dependency review, secret scanning, IaC scanning, container scanning, and release SBOM generation.
- Maintain incident, key-rotation, provider-compromise, data-deletion, restore, and customer-notification runbooks.

Data classification and retention

Class	Examples	Controls	Retention
Restricted content	Manuscripts, production text, audio, voice-consent evidence	Private objects, encryption, least privilege, no telemetry body	User/org policy; explicit deletion
Sensitive operational	Provider credentials refs, signed links, billing identifiers, support access	Secret manager/tokenization, redaction, audit	Minimum operational need
Confidential metadata	Project metadata, issues/comments, estimates, usage, approvals	Tenant isolation, encrypted DB/backups	Contract and account lifecycle
Audit/security	Login, role, export, support, release, deletion events	Append-only, content-safe, restricted search	Defined compliance/security term
Public/low sensitivity	Marketing pages, public provider descriptions	Integrity and standard web controls	Product lifecycle

> **Privacy rule:** A manuscript is not ordinary upload data. Treat it as unpublished intellectual property. Default telemetry, support tools, analytics, cache, and model/provider behavior must be evaluated from that assumption.

Rights and synthetic narration governance

- Capture who asserts rights to the manuscript and their authority; do not represent that the platform independently verified ownership.
- For cloned/custom voices, store provider consent identifiers, speaker authorization evidence, allowed uses, territory/time restrictions, and revocation state.
- Classify providers/voices as local-development, BYOK, commercial-hosted, custom-consented, or prohibited for each plan/target.
- Record synthetic narration disclosure and required attribution in project metadata and release manifest.

- Evaluate target policy at release time using a versioned ruleset and verified-on date; separate technical compatibility from policy eligibility.
- Prevent automatic substitution to a voice with different rights or disclosure consequences.
- Provide a takedown/rights complaint process and the ability to freeze/delete affected releases.

Quality engineering strategy

Test pyramid and release evidence

Layer	Coverage	Cadence
Unit	State transitions, authorization policy, normalization, segmentation, estimates, cache hash, profile rules	Fast; every PR
Property/fuzz	Chapter/text edge cases, Unicode, SSML escaping, file names, API schemas	Every PR or nightly by cost
Provider contract	Adapter capabilities, normalized errors, timeouts, retries, provenance using mocks/recorded fixtures	Every PR; limited live canary
Integration	API + Postgres + queue + object store + workers; outbox, leases, cancellation, reconciliation	Every PR for core subset
Audio fixtures	Decode, joins, metadata, M4B chapters, mastering measurements, target packages	Pinned golden fixtures; every PR/nightly
Frontend	Components, accessibility, error states, server-state transitions	Every PR
End-to-end	Create → upload → structure → sample → estimate → produce → review → export with mock provider	Every PR/nightly
Security	AuthZ matrix, IDOR, upload abuse, dependency/SAST/secret/container/IaC scans	Every PR/release
Performance	API latency, queue throughput, large project, provider throttling, worker drain, storage	Nightly/pre-release
Resilience	Kill API/worker/queue connection, retry storms, provider outage, DB failover, restore drill	Pre-release/quarterly
Manual	Keyboard/screen reader, audio editorial review, browser matrix, operational runbook rehearsal	Milestone/release

CI/CD gates

1. Validate repository hygiene, generated-file policy, lockfiles, license policy, and conventional metadata.
2. Backend: format/lint, type check, unit/property tests, integration tests, coverage threshold focused on critical modules.

3. Frontend: format/lint, strict TypeScript build, component tests, accessibility tests, production build.
4. Security: dependency review, SAST, secret scan, container/IaC scan, SBOM generation, action pinning policy.
5. Media: run golden audio fixtures and verify codec, chapters, metadata, checksums, and QC profile outputs.
6. Build versioned immutable images; scan/sign/attest; push once.
7. Deploy to staging; run migrations, smoke tests, full mock-provider E2E, and active-job drain/reconciliation test.
8. Promote the exact staged images to production; monitor canary; support rollback and forward-fix.

Observability specification

Surface	Measure	Rule
API	request rate, status, latency, auth failures, rate limits, payload class	Do not log manuscript bodies or signed URLs
Queue/jobs	queue depth/age, job duration, unit states, retries, orphaned leases, cancellation latency	Alert on age/stuck growth, not just failures
Providers	availability, latency, throttle, error class, billed units, model/voice	Separate provider degradation from internal defects
Audio	decode failures, processing time, analyzer drift, QC pass rate, package failure	Tag profile/tool version
Cost	estimate vs actual, reservation utilization, cost per approved hour/title, cache savings	Currency and price snapshot required
Storage	bytes by artifact class/age/org, lifecycle backlog, orphan objects, egress	Avoid tenant IDs in high-cardinality public metrics
Product	activation, sample approval, production completion, review/rework, export, retention	Privacy-preserving events; no content
Security	suspicious auth, cross-org denials, upload rejects, admin/support access, secret/scan findings	Route to incident process

Definition of done

- Requirement and acceptance criteria are linked to implementation and tests.
- Authorization, state transition, idempotency, failure, and retry behavior are covered - not only the happy path.

- Database migrations, metrics/logs/traces, feature flags, docs/runbooks, and support impact are included.
- No new manuscript/audio content enters telemetry or analytics.
- User-facing error behavior and accessibility are verified.
- Audio-affecting changes update or intentionally preserve fixture expectations and profile/tool versioning.
- The feature is exercised in a production-like environment and is safe to roll back or disable.

Delivery Roadmap and Backlog

Sequencing model

The roadmap is organized by risk retirement, not by visual excitement. Dates depend on team size and existing infrastructure. The durations below assume roughly two backend/platform engineers, one frontend engineer, a product designer/manager, and part-time audio/QA/security support. A smaller team should preserve the sequence and reduce concurrent scope.

Phase	Theme	Duration	Scope	Exit gate
Phase 0	Repository rescue	1-2 weeks	Clean build/tests; correct Vite/CI; ignore rules; lockfile; production config; baseline docs	Reproducible commit and CI green
Phase 1	Production foundation	3-5 weeks	Auth/org/RBAC, Postgres, migrations, object storage, durable queue, jobs, structured errors/logging	Restart-safe private job with mock provider
Phase 2	Studio MVP	6-10 weeks	Projects, manuscript versions, structure editor, voice samples, pronunciations, estimates, durable production, selective regeneration	Pilot user completes controlled title
Phase 3	Quality and release	5-8 weeks	Mastering/QC profiles, review/issues/approvals, cover/metadata, chapter/MP3/M4B packages, manifests	Release candidate passes internal target profile
Phase 4	Commercial operations	4-7 weeks	Plans/quotas/usage ledger, billing hooks, retention, webhooks, admin/support, SLO dashboards	Paid pilot with unit economics and support readiness
Phase 5	Scale and growth	Ongoing	Templates, portfolio, localization, distribution connectors, enterprise SSO, additional licensed providers	Evidence-based expansion metrics

First 72 hours: repository rescue

1. Create frontend/index.html from the current public/index.html and confirm npm run build from a clean checkout.
2. Move ci-cd/github-actions.yml to .github/workflows/ci.yml and ensure branch protection requires it.
3. Add root/frontend/backend .gitignore and .dockerignore rules; remove .DS_Store, caches, bytecode, uploads, outputs, and synthesized cache from source.
4. Commit exactly one frontend lockfile; replace npm ci || npm i with deterministic npm ci.
5. Replace the hardcoded API URL with validated VITE_API_BASE_URL or a same-origin reverse-proxy path.
6. Add Ruff/formatting and Python type-check baseline; ESLint/Prettier/Vitest baseline; run both builds/tests in CI.

7. Run the backend under Gunicorn (or selected production server) and the frontend as static assets/reverse-proxied service; do not ship dev/preview servers.
8. Drop container root, add health checks, init/signal handling, nonsecret environment validation, and persisted data policy.
9. Document verified supported file types and explicitly reject everything else on the server.
10. Publish a current-state README that labels local prototype limitations honestly and links to this blueprint.

Foundation epics and exit criteria

Epic	Deliverables	Exit criterion
FND-1 Identity and org	OIDC/session, organizations, memberships, RBAC middleware, authz tests	Two orgs cannot read or mutate each other's projects/assets in API/E2E tests
FND-2 Data layer	Postgres schema, SQLAlchemy, Alembic, transaction patterns, optimistic versioning	Migrations up/down strategy tested; concurrent edit conflict works
FND-3 Object boundary	Quarantine/trusted buckets, signed upload/download, checksums, lifecycle, artifact metadata	API containers can restart without losing files; raw keys never exposed
FND-4 Durable jobs	Queue, Job/Unit/Attempt tables, leases, heartbeats, retry/cancel/reconcile, outbox	Kill worker/API mid-job; system resumes or reaches explicit recoverable state
FND-5 Provider contract	Mock provider, expanded adapter schema, error normalization, capability/health	Contract suite passes for mock and Polly sandbox adapter
FND-6 Telemetry	Correlation IDs, structured logs, metrics, traces, error codes, dashboards	One job traceable end-to-end without manuscript content in telemetry
FND-7 Delivery pipeline	CI gates, immutable images, staging deploy, migrations, smoke, rollback	Exact staging artifact promoted; rollback/drain rehearsal documented

Pilot release backlog

Priority	Backlog item	Stream	Why now
P0	Project/org schema and authorization boundary	Foundation	No user-facing work before tenant safety
P0	Signed upload + quarantine + file allowlist/MIME validation	Foundation	Manuscripts leave API filesystem
P0	Durable Job/JobUnit/Attempt engine with mock provider	Foundation	Restart/cancel/retry demonstrable
P0	Artifact lineage and object-store abstraction	Foundation	Every result is immutable and traceable
P0	Environment-driven web API base and production web serving	Repo	Deployable frontend
P0	Project portfolio and resumable project shell	Web	Browser refresh loses no workflow state

Priority	Backlog item	Stream	Why now
P0	Manuscript version + async parser + structure review	Studio	User confirms chapters
P0	Voice capability search + sample job + approval	Studio	No full run before playable sample
P0	Estimate + budget approval + usage reservation	Commercial	Paid calls controlled
P0	Production graph + progress + cancel/retry	Studio	Long job safely operable
P0	Versioned cache hash and selective invalidation	Studio	One edit avoids full rerun
P0	Chapter assembly/master artifact + decode validation	Audio	Reproducible chapters
P0	QC profile engine and initial target profile	Quality	Objective measurements and blockers
P0	Release approval + chapter MP3/M4B + manifest	Release	Immutable downloadable candidate
P1	Pronunciation dictionary with test sample	Editorial	Common name/acronym corrections
P1	Review issues anchored to playback/text	Editorial	Correction loop documented
P1	Cover and metadata validation	Release	Package completeness
P1	In-app/email milestones	Operations	Action-required states visible
P1	Admin provider/queue/job/support view	Operations	Support without shell access
P1	Plan entitlements and usage dashboard	Commercial	Pilot economics visible
P1	Retention/deletion workers and evidence	Governance	Lifecycle promise enforceable
P2	Organization templates and portfolio reporting	Growth	Repeat-title efficiency
P2	Signed webhooks	Integration	Publisher workflow integration
P2	Additional licensed TTS provider	Provider	Proves portability after contract matures

Release gates

Gate	Required evidence	Audience
Alpha	Clean build; private org projects; mock-provider durable job; safe upload; restart recovery; no paid customer data	Internal team
Design partner	Sample/estimate/production/review/export works; provider spend controlled; support/runbooks; target profile labeled	3-5 invited projects
Paid pilot	Usage ledger/billing hooks; retention; SLO dashboards; incident path; backup/restore drill; policy/terms review	Limited paid cohort
General availability	Capacity/load evidence; security review; WCAG core-flow audit; provider fallback/degradation; support SLA; unit economics	Defined public segment

Team operating model

- One accountable product lead owns requirements, sequencing, target user evidence, and commercial metrics.
- One technical lead owns architecture boundaries, state/data integrity, security posture, and release quality.
- An audio domain owner validates profiles, fixtures, listening workflow, and platform changes; this cannot be inferred from code alone.
- Design works from complete state models: loading, empty, stale, retry, over-budget, policy-ineligible, cancelled, and approval-stale states.
- Security/privacy review occurs during design for upload, auth, provider credentials, support access, retention, and rights - not just before launch.
- Every milestone is demonstrated with a realistic book-length fixture and a forced failure/recovery scenario.

Prosperity Operating System

The operating loop

1. Acquire a narrow user with a completed manuscript and a real production deadline.
2. Activate on a playable, approved sample - not account creation.
3. Convert when the user approves a transparent estimate and starts durable production.
4. Deliver an approved release package with measurable quality evidence.
5. Learn from rework: chapter edits, pronunciation corrections, retry causes, QC failures, support touches, and cost variance.
6. Retain through the next title, backlist batch, organization template, or editor collaboration.
7. Expand only when repeatability and gross margin are demonstrated.

Metric tree

Branch	Primary metric	Required segmentation
Acquisition	Qualified manuscript projects created	Source, user segment, book status, target
Activation	Valid upload → confirmed structure → approved sample	Time and fallout at each step
Conversion	Current estimate approved and production started	Price/provider/length and abandoned reason
Production	Completion rate and time to first/last approved chapter	Queue, provider, retries, rework
Quality	First-pass QC, issues per finished hour, regeneration rate, waiver rate	Profile, provider/model, chapter type
Delivery	Release candidates produced and downloaded/published	Target and eligibility state
Retention	Second active title within 90/180 days; active studio months	Individual versus organization
Economics	Gross margin per title/finished hour; estimate variance; support cost	Plan, provider, cache reuse, storage/egress
Reliability	Job success, orphan recovery, provider degradation, SLO/error budget	Environment/provider/worker type
Trust	Security incidents, deletion SLA, rights complaints, support-access events	Severity and root cause

Initial experiments

ID	Hypothesis	Measure	Decision rule
E-01 Guided sample	A fixed 3-step path improves approved samples	Approval rate and time versus open configuration	Keep if +20% activation without more support
E-02 Estimate clarity	Provider cost + cache savings + overage cap increases production starts	Estimate-to-start conversion, refund/support questions	Keep if conversion rises and disputes fall
E-03 Pronunciation before run	Prompting top names/acronyms before production reduces regeneration	Corrections/finished hour and incremental cost	Keep if meaningful rework reduction
E-04 Studio template	Publisher templates reduce setup time on title two	Time-to-sample for repeat org projects	Promote to paid differentiator if repeat gain
E-05 Human QA add-on	Optional paid proof-listening accelerates trust and learning	Attach rate, margin, issue classes, NPS	Scale only with standardized SLA
E-06 BYOK vs included usage	Different segments prefer control versus convenience	Conversion, margin, support, provider failure	Segment plans; do not force one model
E-07 Target profile messaging	Evidence-based 'ready for profile X' beats generic quality claims	Export conversion and support/rejection reports	Expand profile library after proof

Unit economics framework

The usage ledger should make every finished title explainable. A simplistic provider character bill is not the full cost. The model below becomes a dashboard and a pricing review input.

Component	Calculation	Evidence
Provider synthesis	Billed characters/seconds by engine/model plus retries not credited	Attempt/usage ledger
Compute	Ingest, audio decode/join/master/QC, reconciliation	Worker time and infrastructure allocation
Storage	Source, segments, masters, packages, backups by class and age	Artifact inventory
Egress	Downloads, provider transfers, distribution handoffs	Cloud/network usage
Support/QA	Human minutes, incidents, refunds, proof-listening	Support and service tracking
Revenue	Subscription allocation, usage fee, add-on services, credits/refunds	Billing/ledger
Gross margin	Revenue minus variable provider/compute/storage/egress/service cost	Per title, finished hour, org, plan

Growth without self-destruction

- Start with completed, edited manuscripts. Early-stage writing tools create a different product and weaker production intent.
- Sell the sample-to-release workflow, not an unlimited generation promise.
- Use design partners to build templates and profiles around repeated real problems, then productize them.
- Keep paid provider usage visibly metered. Give customers budgets and estimates instead of hiding cost until an invoice.
- Treat support work as product research: classify every intervention and automate the recurring safe cases.
- Add a provider only when there is demonstrated demand for language, quality, rights, price, or resilience that the current catalog cannot meet.
- Add distribution integration only after package generation and policy eligibility are reliable; automation should not accelerate bad releases.
- Do not discount security/privacy as enterprise polish. Unpublished manuscripts make trust a core buying criterion even for individual authors.

Risk register

ID	Risk	Likelihood	Impact	Mitigation	Owner
R-01	Provider terms do not permit intended hosted/commercial use	High	Critical	Approved catalog, legal review, BYOK option, no silent provider change	Product + Legal
R-02	Synthetic narration rejected or restricted by target	High	High	Versioned target policy, disclosure, eligibility gate, target-specific package	Product
R-03	Cost spike from retries/abuse/long books	Medium	Critical	Estimate, reservation, quotas, idempotency, anomaly alerts, kill switch	Engineering + Finance
R-04	Manuscript leak or cross-tenant access	Low/Med	Critical	RBAC, isolation tests, signed objects, safe logs, encryption, incident plan	Security
R-05	Audio passes simple checks but sounds bad	High	High	Objective profiles plus human approval and fixture listening	Audio + Product
R-06	Provider/model changes alter voice output	High	High	Provenance, version pinning where possible, new sample/approval on change	Provider owner
R-07	Large/hostile documents exhaust parsers	Medium	High	Quarantine, budgets, sandbox, async processing, rejection	Security + Platform
R-08	Team overbuilds architecture before product proof	Medium	High	Modular monolith, phase gates, extraction criteria, metric-driven scale	Tech lead
R-09	Users expect guaranteed retailer acceptance	High	Medium	Precise claims, verified-on date, manifest, disclaimers, update process	Product + Support
R-10	Deletion/retention promises fail across caches/backups	Medium	High	Data map, lifecycle workers, restore/delete drills, evidence	Platform + Privacy

ID	Risk	Likelihood	Impact	Mitigation	Owner
R-11	Pricing hides poor gross margin	Medium	High	Per-title ledger, cohorts, usage separation, price experiments	Product + Finance
R-12	Automated structure/normalization damages text	Medium	High	Confidence, user confirmation, immutable source, diff and rollback	Editorial

Decision dashboard for leadership

Question	Evidence threshold	If below threshold
Do users approve samples?	≥40% of valid uploads reach approved sample	Fix onboarding/voice value before adding providers
Do approved samples convert?	≥50% approve estimate/start within pilot	Revisit quality, cost, trust, or target positioning
Can production finish reliably?	≥95% jobs complete without manual operator repair	Do not scale acquisition; invest in durability
Does selective regeneration work?	Most edits regenerate <10% of book units	Fix version/invalidation graph
Are estimates credible?	Actual provider cost within ±10% absent user change	Fix billing model before flat pricing
Does quality reduce risk?	Rising first-pass rate and falling issues/finished hour	Improve profiles/direction/preflight
Does the business make money?	Positive contribution margin per title/finished hour by target segment	Adjust pricing/provider/service scope
Do users return?	Meaningful second-title or multi-project organization behavior	Strengthen repeat workflow/templates

Acceptance Framework

Minimum production launch acceptance

Area	Acceptance evidence
Repository	Clean checkout builds/tests without generated artifacts; CI runs from recognized path; lockfiles deterministic
Identity	Every protected route requires authentication; server-enforced org and role isolation passes negative tests
Upload	Signed quarantine upload; allowlisted verified types; bounded scanning/parsing; safe failure
Data	Projects, manuscript versions, jobs, attempts, artifacts, QC, approvals, releases, usage, and audit persist in DB/object store
Jobs	Restart recovery, retry/backoff, cancellation, idempotency, leases, reconciliation, and provider rate limits demonstrated
Cost	Estimate, budget approval/reservation, actual usage, cache savings, and overage policy demonstrated
Audio	Provider bytes validated; selective regeneration; loss-minimizing masters; versioned assembly/mastering/QC
Review	Anchored issues, regeneration impact, QC blockers/waivers, exact-version approvals
Release	Target-specific immutable package, cover/metadata validation, manifest/checksums, signed download
Operations	Structured content-safe telemetry, dashboards/alerts, runbooks, backup/restore and deploy/rollback rehearsal
Security	Threat model, ASVS-based review, dependency/secret/SAST/container/IaC gates, support access audit
Accessibility	WCAG 2.2 AA core flows verified by automated and manual checks
Policy	Provider commercial classification, rights declaration, voice consent where applicable, target eligibility/disclosure
Commercial	Entitlements, quotas, usage visibility, support path, retention/deletion policy, pilot terms

Go/no-go questions

- Can a worker and API be killed during a book and recover without silent loss or duplicate uncontrolled spend?
- Can a user from Organization A access any Organization B project, status, audio, signed link, cache result, cost, or audit event?

- Can a malformed or oversized document exhaust the API or escape the parser boundary?
- Can a paid provider call occur without a current estimate approval, quota, reservation, and usage trail?
- Can an old approval remain valid after text, voice, pronunciation, mastering, QC profile, cover, or target changes?
- Can the team reproduce a released package and prove its exact source/config/toolchain?
- Can support resolve a job without reading the manuscript or opening a shell on production?
- Can leadership see contribution margin and support cost per title rather than only subscriptions and provider spend?

Architecture Decision Records to create

ADR	Decision
ADR-001	Modular monolith and extraction criteria
ADR-002	System of record and transaction/outbox pattern
ADR-003	Durable job engine, leases, idempotency, and reconciliation
ADR-004	Object storage layout, quarantine, encryption, lifecycle, and signed access
ADR-005	Tenant-scoped artifact cache and invalidation graph
ADR-006	Provider adapter, commercial policy, BYOK, and model/voice provenance
ADR-007	Immutable manuscript/structure/chapter/config versioning
ADR-008	Audio intermediate, mastering, QC, and target profile versioning
ADR-009	Authentication, organization RBAC, and support access
ADR-010	Usage reservation, ledger, quotas, and billing boundary
ADR-011	Observability content policy and data minimization
ADR-012	Retention, deletion, backup expiry, and legal holds

Appendix A - Suggested configuration objects

Synthesis configuration

Field	Meaning
schema_version	Version of configuration schema
provider/model/voice	Stable adapter identifiers plus provider-native values
locale/language	BCP 47 where applicable
controls	Rate, pitch, style, volume and supported vendor extensions
normalization_profile_version	Text transformation recipe
pronunciation_set_version	Resolved lexicon version
segmenter_version	Semantic segmentation logic
ssml_renderer_version	Provider-safe markup generation
output_request	Provider codec/sample preferences
policy_class	Commercial/BYOK/local/custom-consented classification

QC rule result

Field	Meaning
rule_id / profile_version	Stable definition of the evaluated rule
artifact_id / checksum	Exact file evaluated
measurement	Numeric/string value and unit
threshold	Operator/range/enumeration used for this target
method	Analyzer name/version and measurement window
severity	Blocking, warning, or informational
result	Pass, fail, not-applicable, error
evidence	Safe analyzer output/reference
remediation	Actionable explanation
waiver	Actor, role, reason, time, and affected release

Release manifest

Section	Required content
release	Release ID/version, project ID, target, created/approved times
bibliographic	Title, author, narrator/voice attribution, language, publisher, identifiers
source	Manuscript, structure, chapter, normalization, pronunciation, direction versions/checksums
production	Provider/model/voice provenance and billed-unit summary
audio profile	Assembly, mastering, QC, analyzer, and export profile versions
files	Relative name, kind, checksum, bytes, duration, codec, sample rate, channels, bitrate
quality	QC summary, blockers, waivers, consistency result
approvals	Sample/QC/release approval identities and bound hashes
rights/policy	Declarations, consent references, synthetic disclosure, eligibility rule version
lineage	Parent release and changed components

Appendix B - API behavior examples

Stable error envelope

```
{
  "error": {
    "code": "BUDGET_APPROVAL_STALE",
    "message": "The approved estimate no longer matches the current production
configuration.",
    "correlation_id": "req_...",
    "retryable": false,
    "field_errors": [],
    "next_actions": ["CREATE_ESTIMATE", "REQUEST_APPROVAL"]
  }
}
```

- Use 409 Conflict for stale versions/state conflicts; 422 for schema/semantic validation; 429 for rate/quota conditions; 503 for temporary provider/system degradation.
- Never encode retry behavior only in prose. retryable and next_actions let the client present safe choices.
- Correlation IDs are safe opaque identifiers. Internal logs map them to detailed diagnostics.

Idempotent production start

1. Client creates a unique organization-scoped idempotency key and submits the current estimate_id.
2. Server locks/evaluates current project configuration, estimate, approval, quota, and existing idempotency record.
3. Server creates usage reservation, job, job units, and outbox event in one transaction.
4. Duplicate request with the same key and equivalent payload returns the same job.
5. Duplicate key with a materially different payload returns a conflict rather than starting new spend.

Appendix C - Source-informed cleanup checklist

Change	Proof
Move frontend/public/index.html to frontend/index.html	Vite build entry exists at the required root
Move ci-cd/github-actions.yml to .github/workflows/ci.yml	GitHub recognizes CI
Remove .DS_Store, __pycache__, .pytest_cache, uploads, outputs, cache	Only source/fixtures remain
Add .gitignore/.dockerignore	Generated/private files cannot re-enter commits/build contexts
Commit package-lock.json	npm ci is deterministic
Use VITE_API_BASE_URL or /api same-origin	Deployment no longer calls visitor localhost
Replace Python/Vite dev serving	Production process model
Persist/replace active_tasks	Restart-safe jobs
Persist outputs/cache appropriately	Container replacement does not erase work
Add upload server allowlist/MIME verification	Frontend accept is not the security boundary
Reduce/scope CORS	Only intended origins
Add auth and ownership	Task ID no longer authorizes access
Add retry/cancel/queue limits	Safe external-provider execution
Replace print with structured logging	Actionable, content-safe operations
Harden containers	Non-root, health, resource/signal policy
Expand tests and scans	Merge/release confidence

Appendix D - Reference standards and live sources

These sources inform the blueprint. Platform pricing, limits, narration policy, and submission rules are live dependencies. The product should store a verified-on date for profile/policy rules and re-check authoritative sources before release claims.

- Current source README
- ACX audio submission requirements
- Spotify for Authors - self-published authors
- Spotify direct audiobook publishing announcement
- Amazon Polly quotas and limits
- Amazon Polly pricing
- Apple Books Asset Guide
- OWASP Application Security Verification Standard
- WCAG 2.2
- NIST Secure Software Development Framework
- OpenAPI Specification 3.1

Final directive

> **Build this:** A private, durable, revision-safe audiobook production studio whose outputs are measurable, explainable, and commercially governable. The existing prototype proves the happy path. Prosperity comes from owning the difficult path: changes, failures, quality, rights, cost, approvals, and repeat production.